CMPT 354 - Mini Project Report

2. Project Specifications

- The rest of the definitions of the domain is your work.
    - **Member**:
        - a list of people that is a user/member of the library.
        - each tuple represents a member, contains:
            - an unique **memberID**,
            - **name** of this person,
            - fines in total (**balance**).
                - positive = amount needed to pay in total
                - if paid, the paid amount will be subtracted from this balance.
                - zero = no need to pay
            - **Contact** Information : phone no.
        - **Member**(<u>memberID</u>, memberName, balance, phone).

    - **Library Items**:
        - A list of all library items.
        - each tuple represents an item contains:
            - an unique **itemID**,
            - **item type** of what this item is
              (e.g. printed_book, magazine, … etc),
            - **status** of this item in library
                - 'available' = able to borrow
                - note: online books have infinite amounts, always available.
                - 'borrowed' = unable to borrow
                - 'purchasing' = might be added to library in the future
            - **Title** of this item.
            - the fine amount when return after the due date (**cost**)
        - **Item**(<u>itemID</u>, itemType, itemStatus, title, cost).

    - **Books**:
        - A list of books.
        - Each tuple represent a book
        - library Item with type as either printed or online books:
            - **item** id that identifies which book
            - the **ISBN** for this book.
        - Because only books have ISBN, magazines don't
        - **Books**(<u>itemID</u>$^{\text{FK-Item}}$, ISBN).

- **Author:**
  - A list of authors,
  - Each tuple represent an author, contains:
    - an unique **authorID** for each author
    - **Name** of this author
  - **Author**(<u>authorID</u>, authorName).

- **Item_Author:**
  - A list of which author(s) contributes to which items
  - each tuple represents an author's contribution to an item
    - **itemID** that identify which item (fk Item)
    - **authorID** that identify who's the contribute author. (fk author)
  - **Item_Author**(<u>itemID</u>$^{\text{FK-Item}}$, <u>authorID</u>$^{\text{FK-Author}}$).

- **Borrowing Records**:
  - A record for all borrowing history. (both returned, not returned)
  - Each tuple represents 1 borrowing history.
  - When a member borrows an item from the library,
    each records will have
    - an unique **borrowID**,
    - the **item** borrowed (identify by itemID, fk to Item relation)
    - the **member** borrowing (identify by memberID, fk to the Member relation)
    - the **borrow date**
    - the **due date**.
    - the **return date**.
  - **Borrow**(<u>borrowID</u>, itemID, memberID, borrowDate, dueDate, returnDate).

- **Payment status Records**:
  - A list of payment history (both paid, unpaid)
  - Each tuple represents an overdue return.
  - When an borrow is over the due date, each records will have:
    - an unique **paymentID**,
    - the **borrow record** that (identify by borrowID, fk of Borrowing relation),
      - identify what this payment is for which borrowing
    - the **payment status**
      - 'paid' = this payment record has been paid already
      - 'unpaid' = this payment hasn't been paid
    - the **payment date** that this member pays this payment
  - **Payment**(<u>paymentID</u>, borrowID$^{\text{FK-Borrow}}$, payStatus, payDate).

- **Library Events**:
  - A list of library events
  - each tuple represents an event hold by the library that contains:
    - an unique **eventID**,
    - the main **organizer** of this event, (identify by the staffID, fk of Staff relation)
    - the **name** of this event.
    - **event type**
      (e.g. book_clubs, art_shows, …etc),
    - the **date, time, location** that this event will be hold,
  - **Events**(<u>eventID</u>, staffID$^{FK\text{-}Staff}$, eventType, eventName, date, time, location).

- **Event_Type**:
  - A list for connecting event with target audience(s).
  - Each tuple represents an event's corresponding audience
    - the **eventID** to identify the event. (fk Events)
    - the target **audience** of this event.
      (e.g. 'child', 'teens', seniors, … etc)
  - **Event_Audience**(<u>eventID</u>$^{FK\text{-}Events}$, <u>audience</u>).

- **Event_Attendance**:
  - A list for connecting what member are attending what event
  - Each tuple represents a member attending an event
    - the **eventID** to identify the event. (fk Events)
    - the **memberID** to identify who is attending. (fk Member)
  - **Event_Attendance**(<u>eventID</u>$^{FK\text{-}Events}$, <u>memberID</u>$^{FK\text{-}Members}$).

- **Event_Helpers**:
  - A list for connecting who is helping for an event
  - Each tuple represent an staff helping an event
    - the **eventID** to identify the event
    - the **staffID** to identify who si helping.
  - **Event_Helpers**(<u>eventID</u>$^{FK\text{-}Events}$, <u>staffID</u>$^{FK\text{-}Staff}$).

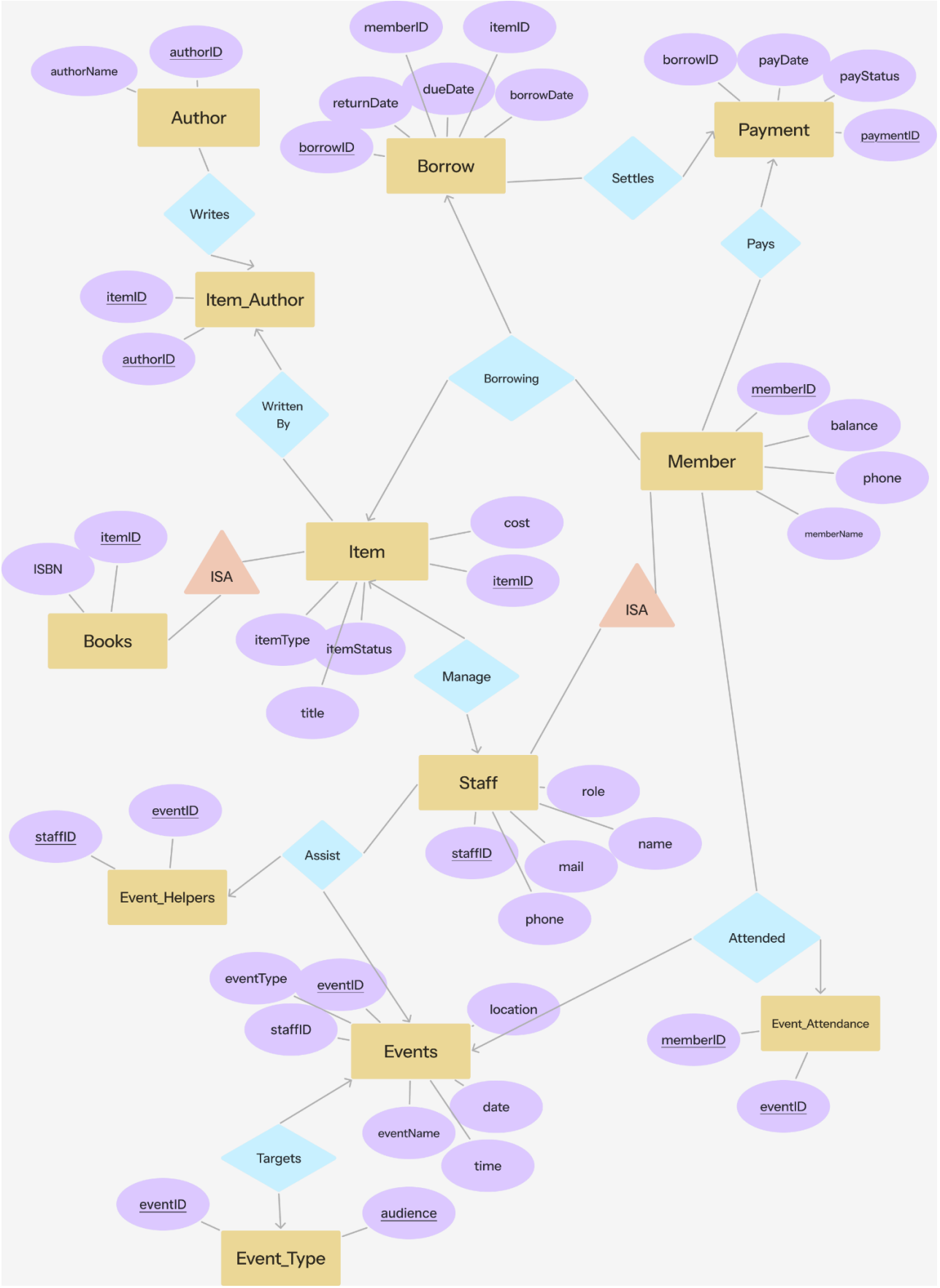
- **Library Personnel**:
  - A list of library staffs
    - an unique **staffID**,
    - the **name, phone** no. , **mail** address of the staff.
    - the **role** of this staff
      (e.g. 'Administrator', 'Librarian', 'event_coordinator'')
  - **Staff**(<u>staffID</u>, name, phone, mail, role).

- Find out about needs and requirements with simple words
    - **Member** v.s. **Library Items (1:M)**
        - 1 **Member** can borrow many **Items**.
        - 1 **Items** can be borrowed by 1 **Member** at a time.
    - **Member** v.s. **Events (1:M)**
        - 1 **Member** can attend many **Events**.
        - 1 **Events** can have many **Members**.
    - **Item** v.s. **Book (1:1)**
        - 1 **Item ID** can have 1 **Book ISBN**.
        - 1 **Book ISBN** can have 1 **Item ID**.
    - **Item** v.s. **Author (1:M)**
        - 1 **Item** with category as book can have many **Authors**.
        - 1 **Author** can contribute to many book **Items**.
    - **Member** v.s. **Payment (1:M)**
        - 1 **Member** can have many **Payments**
        - 1 **Payment** is associated with 1 **Member**.
    - **Payment** v.s. **Borrow (1:M)**
        - 1 **Payment** settles 1 late returned **Borrow** Item.
        - 1 **Borrow** items can have many **Payments**. (Many people have borrowed this item and not returned on time)
    - **Staff** v.s. **Item (M:N)**
        - Many **Staff** members manage many **items**.
        - Many **items** can be managed by many **Staff** members.
    - **Staff** v.s. **Events (1:M)**
        - 1 **Staff** can organize many **events**.
        - Each **Event** can be organized by 1 **Staff** member.
    - **Events** v.s. **Event_Audience (1:M)**
        - 1 **Event** can have many **Event_Audience**.
        - Each **Event_Audience** can be the target of many **Events**.
    - **Events** v.s. **Event_Attendance (M:N)**
        - 1 **Event** can have many **members** attending.
        - 1 **member** can attend many **Events**.
    - **Events** v.s. **Event_Helper (M:N)**
        - 1 **Event** can have many **staff** members helping.
        - 1 **staff** can help with many **Events**.

3. E/R Diagram

This Entity-Relationship (E/R) model represents a library management system incorporating above listed entities their attributes and purpose.

https://www.figma.com/design/1nvRcK9Vt2e9qBWE6vKtuV/Untitled?node-id=0-1&t=ur1SOJCQaqOOiWVI-1



4. anomalies

- Functional Dependencies
  - **Member** : memberID → memberName, phone, balance.
    - No anomalies. Each member has unique information directly tied to memberID.
  - **Item** : itemID → itemType, itemStatus, title, cost.
    - No anomalies. Each item has unique information tied to itemID.
  - **Item_Author** : authorID, itemID → authorID, itemID
    - No anomalies. Reducing redundancy by linking tables.
  - **Books** : itemID → ISBN. (ISBN are unique ISBN → itemID)
    - No anomalies. Both itemID and ISBN are candidate keys. No non-trivial FDs outside these keys.
  - **Author** : authorID → authorName.
    - No anomalies.
  - **Borrow**: borrowID → itemID, memberID, borrowDate, dueDate, returnDate.
    - No anomalies. Each borrow transaction is uniquely identified by borrowID.
  - **Payment** : paymentID → borrowID, payStatus, payDate.
    - No anomalies.
  - **Events** : eventID → staffID, eventType, eventName, date, time, location.
    - No anomalies, staffID represents the one who is responsible and organizes the event.
  - **Event_Audience**: eventID, audience → eventID, audience.
    - No anomalies. Reducing redundancy by linking tables. Composite key with no non-key attributes depend directly on it.
  - **Staff** : staffID → staffName, role.
    - No anomalies.
  - **Event_Helpers** : eventID, staffID → eventID, staffID
    - No anomalies. Reducing redundancy by linking tables.
- No anomalies
  - Eliminated Insert/Delete/Update Anomalies:
    - Each table represents a single entity preventing data redundancy.
    - We can insert information about an entity without needing to insert related information about other entities unless required by relationships (e.g. to create Borrow record, we need existing memberID and itemID). For example, to insert a new member, we only need memberID and the other attributes for the member table. We don't require information for item, borrow or payment to add a new member. Similarly we check for every relation if this is the case.
    - Deleting an entity record does not unintentionally delete other related information about other entities. If we delete a member, we only lose information about the members and we do not accidentally lose data for staff members etc.. Another example, deleting an event only removes event details, not staff information.

    - Then we check for updating an attribute in a table. For instance, if a member's phone number changes, we only update it in the Member table. So, we do not need to search for and update phone numbers on other tables.

- Normalization (1NF→2NF→3NF→BCNF)
    - 1NF: We will make sure all attributes are indivisible and there are no repetitions. M:N relationships were resolved via linking tables. Most attributes are indivisible (single-attribute primary key) and linking tables have a composite key.
    - Schema in 2NF: Each table is in 2NF because each table, every non-key attribute is fully dependent on the primary key of that table. For linking tables with composite keys, there are no non-key attributes to check for partial dependencies.
    - 3NF: Every relation is 2NF and every non-prime attribute is non-transitively dependent on the primary key. There are no transitive dependencies across all relations. For example, in Member(memberID, memberName, phone, balance), memberName, phone and balance are directly dependent on memberID, and not indirectly through another non-key attribute. The same holds true for other tables. Thus, the schema satisfies 3NF.
    - BCNF: Every determinant (LHS) in each table must be a candidate key. In every table the primary key is indeed the only determinant, and therefore a superkey and a candidate key. Therefore, the schema does satisfy BCNF.